10: Kubernetes storage

Storage is critical to most real-world production applications. Fortunately, Kubernetes has a mature and feature-rich storage subsystem called the *persistent volume subsystem*.

The chapter is divided as follows:

- The big picture
- Storage providers
- The Container Storage Interface (CSI)
- The Kubernetes persistent volume subsystem
- Dynamic provisioning with Storage Classes
- Hands-on

Kubernetes supports lots of storage back-ends, and each requires slightly different configuration. The examples in this chapter are designed to work on Google Kubernetes Engine (GKE) clusters and will not work on other cluster types. The principles and theory that you'll learn is applicable to all types of Kubernetes, it's just the examples that only work on GKE.

The big picture

Kubernetes supports lots of types of storage from lots of different places. For example, block, file, and object storage from a variety of external systems that can be in the cloud or your on-premises datacenters. However, no matter what type of storage, or where it comes from, when it's exposed on Kubernetes it's called a *volume*. For example, Azure File resources surfaced in Kubernetes are called *volumes*, as are block devices from an HPE 3PAR array, and object storage from Alicloud.

Figure 10.1 shows the high-level architecture.

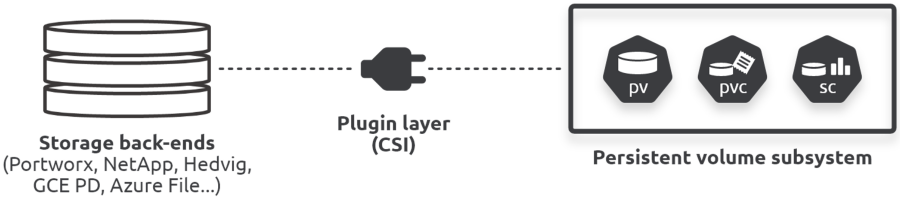

Figure 10.1

On the left are storage providers. They can be traditional enterprise storage arrays from established vendors like EMC and NetApp, or they can be cloud storage services such as AWS Elastic Block Store (EBS) and GCE Persistent Disks (PD). All that's required is a *plugin* allowing their storage resources to be surfaced as volumes in Kubernetes.

In the middle of the diagram is the plugin layer. In simple terms, this is the glue that connects external storage with Kubernetes. Modern plugins are be based on the ***Container Storage Interface (CSI)*** which is an open standard aimed at providing a clean storage interface for container orchestrators such as Kubernetes. If you're a developer writing storage plugins, the CSI abstracts the internal Kubernetes machinery and lets you develop *out-of-tree*.

> **Note:** Prior to the CSI, all storage plugins were implemented as part of the main Kubernetes code tree (*in-tree*). This meant they had to be open-source, and all updates and bug-fixes were tied to the main Kubernetes release-cycle. This was a nightmare for plugin developers as well as the Kubernetes maintainers. However, now that we have the CSI, storage vendors no longer need to open-source their code, and they can release updates and bug-fixes against their own timeframes.

On the right of Figure 10.1 is the Kubernetes persistent volume subsystem. This is a set of API objects that enable applications to consume storage. There are a growing number of storage-related API objects, but the core ones are:

- Persistent Volumes (PV)
- Persistent Volume Claims (PVC)
- Storage Classes (SC)

Throughout the chapter, we may refer to them by their PascalCase truncated API names – *PersistentVolume, PersistentVolumeClaim, and StorageClass.*

PVs are mapped to external storage assets, PVCs are like tickets that authorize applications (Pods) to use them, and SCs make it all dynamic.

Consider the quick example in Figure 10.2.

A Kubernetes cluster is running on AWS and the AWS administrator has created a 25GB EBS volume called "ebs-vol". The Kubernetes administrator creates a PV called "k8s-vol" that links back to the "ebs-vol" via the `ebs.csi.aws.com` CSI plugin. While that might sound complicated, it's not. The PV is simply a way of representing the external storage asset on the Kubernetes cluster. Finally, the Pod uses a PVC to claim access to the PV and start using it.

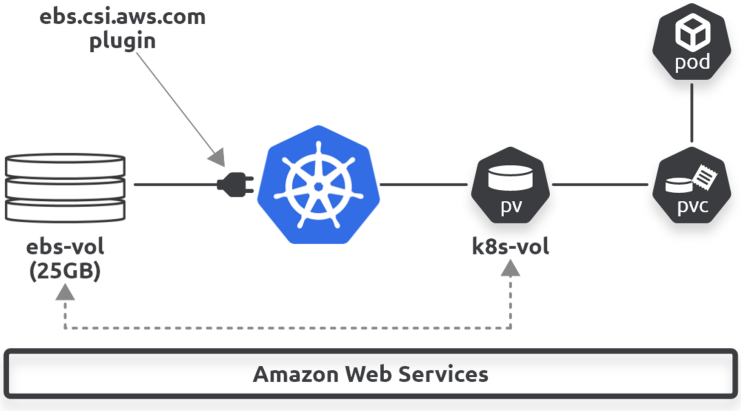

Figure 10.2

A few things worth noting.

1. This was a very manual process that involved the AWS administrator. StorageClasses make this automated.
2. There are rules preventing multiple Pods accessing the same volume (more on this later).
3. You cannot map an external storage volume to multiple PVs. For example, you cannot have a 50GB external volume that has two 25GB Kubernetes PVs each using half of it.

Let's dig a bit deeper.

Storage Providers

Kubernetes uses storage from a wide range of external systems. These can be native cloud services such as *AWS Elastic Block Store* and *Azure File*, but they can also be traditional on-premises storage arrays providing `iSCSI`, `FC` and `NFS` volumes. Other options exist, but the take-home point is that Kubernetes gets its storage from a wide range of external systems including battle-hardened enterprise-grade systems from all the major data management companies.

Some obvious restrictions apply. For example, you can't use AWS storage services if your Kubernetes cluster is running in Microsoft Azure.

Each provider (a.k.a provisioner) needs a CSI plugin to expose their storage assets to Kubernetes. The plugin usually runs as a set of Pods in the `kube-system` Namespace.

The Container Storage Interface (CSI)

The CSI is a vital piece of the Kubernetes storage jigsaw and has been instrumental in bringing enterprise-grade storage from traditional vendors to Kubernetes. However, unless you're a developer writing storage plugins, you're unlikely to interact with it very often.

It's an open-source project that defines a standards-based interface so that storage can be leveraged in a uniform way across multiple container orchestrators. For example, a storage vendor *should* be able to write a single CSI plugin that works across multiple orchestrators such as Kubernetes and Docker Swarm. In practice, Kubernetes is the focus, but Docker is implementing support for the CSI.

In the Kubernetes world, the CSI is the preferred way to write plugins (drivers) and means that plugin code no longer needs to exist in the main Kubernetes code tree. It also exposes a clean interface and hides all the ugly volume machinery inside of the Kubernetes code (no offense intended).

From a day-to-day perspective, your main interaction with the CSI will be referencing the appropriate CSI plugin in your YAML manifest files, and reading its documentation to find supported features and attributes.

Sometimes we call plugins *"provisioners"*, especially when we talk about Storage Classes later in the chapter.

The Kubernetes persistent volume subsystem

From a day-to-day perspective, this is where you'll spend most of your time configuring and interacting with storage.

You start out with raw storage on the left of Figure 10.3. This *plugs in* to Kubernetes via a CSI plugin. You then use persistent volume subsystem API resources to leverage and use the storage in your apps.

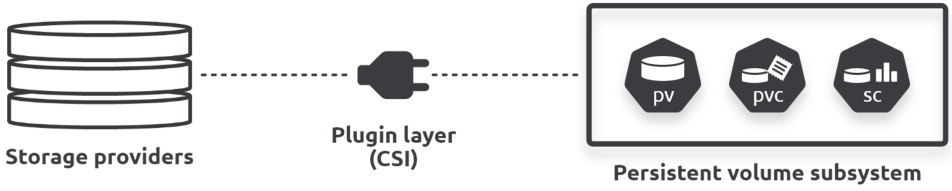

Figure 10.3

The three core API resources in the persistent volume subsystem are:

- Persistent Volumes (PV)
- Persistent Volume Claims (PVC)
- Storage Classes (SC)

Others exist, and storage vendors can extend the Kubernetes API with their own resources to support advanced features.

At a high level, **PVs** are how external storage assets are represented in Kubernetes. **PVCs** are like tickets that grant a Pod access to a PV. **SCs** make it all dynamic.

Let's walk through another example.

Assume you have an external storage system with two tiers of storage:

- Flash/SSD fast storage
- Mechanical slow archive storage

You expect apps on your Kubernetes cluster to use both, so you create two *Storage Classes* and map them as follows.

External tier	Kubernetes Storage Class name
SSD	sc-fast
Mechanical	sc-slow

With the *StorageClass* objects in place, applications can create volumes on-the-fly by creating *Persistent Volume Claims (PVC)* referencing either of the storage classes. Each time this happens, the CSI plugin referenced in the SC instructs the external storage system to create an appropriate storage asset. This is automatically mapped to a PV on Kubernetes and the app uses the PVC to claim it and mount it for use.

Don't worry if it seems confusing, it'll make sense when you go through the hands-on later.

Before doing that, you need to learn a bit more about PVCs and SCs.

Dynamic provisioning with Storage Classes

As the name suggests, storage classes allow you to define different classes/tiers of storage. How you define them is up to you and will depend on the types of storage you have available. For example, if your external storage systems support fast and slow storage, as well as remote replication, you might define these three classes:

- `fast-local`
- `fast-replicated`
- `slow-archive-local`

As far as Kubernetes goes, storage classes are resources in the `storage.k8s.io/v1` API group. The resource type is `StorageClass`, and you define them in regular YAML files that you post to the API server for deployment. You can use the `sc` shortname to refer to them when using `kubectl`.

> **Note:** You can use `kubectl api-resources` to see a full list of API resources and their shortnames. It also shows if the resource is namespaced, its API group, and what its equivalent `kind` is when writing YAML files.

A StorageClass YAML

The following SC defines a class of storage called "fast-local", based on AWS solid state drives (`io1`) in the Ireland Region (`eu-west-1a`). It also requests a performance level of 10 IOPs per gigabyte and encrypted volumes.

```
kind: StorageClass
apiVersion: storage.k8s.io/v1
metadata:
  name: fast-local
provisioner: ebs.csi.aws.com
parameters:
  type: io1
  iopsPerGB: "10"
  encrypted: true
allowedTopologies:
- matchLabelExpressions:
  - key: topology.ebs.csi.aws.com/zone
    values:
    - eu-west-1a
```

As with all Kubernetes YAML, `kind` tells the API server what type of object you're defining, and `apiVersion` tells it which version of the schema to use when creating it. `metadata.name` is an arbitrary string that lets you give the object a friendly name – this example is defining a class called "fast-local". `provisioner` tells Kubernetes which plugin to use, and the `parameters` block lets you finely tune the storage attributes. Finally, the `allowedTopologies` property lets you list where replicas should go.

A few quick things worth noting:

1. StorageClass objects are immutable – this means you can't modify them after they're deployed

2. `metadata.name` should be meaningful as it's how **you** and other objects refer to the class
3. The terms *provisioner* and *plugin* are used interchangeably
4. The `parameters` block is for plugin-specific values, and each plugin is free to support its own set of values. Configuring this section requires knowledge of the storage plugin and associated storage back-end. Each provisioner usually provides documentation.

Multiple StorageClasses

You can configure as many StorageClasses as you need. However, each class can only relate to a single type of storage on a single back-end. For example, if you have a Kubernetes cluster with StorageOS and Portworx storage back-ends, you'll need at least two StorageClasses.

On the flip-side, each back-end storage system can offer multiple classes/tiers of storage, each of which needs its own StorageClass on Kubernetes. A simple example we'll see later is the slower `standard persistent disk` and the faster `SSD persistent disk` tiers offered by the Google Cloud back-end. These are typically implemented with the following SCs on a GKE cluster.

1. `standard-rwo` for the slower standard disk
2. `premium-rwo` for the faster SSD

The following SC defines a block storage volume on a Commvault Hedvig array that is replicated between datacenters in Sunderland and New York. It will only work if you have Commvault Hedvig storage systems and appropriate replication configured on the storage system.

```
kind: StorageClass
apiVersion: storage.k8s.io/v1
metadata:
  name: sc-hedvig-rep
provisioner: io.hedvig.csi
parameters:
  backendType: "hedvig-block"
  rp: "DataCenterAware"
  dcNames: "sunderland,new-york"
```

As you can see, the `parameters` block defines the interesting values and requires knowledge of the plugin and the storage back-end. Consult your storage plugin documentation for details.

Working with StorageClasses

The basic workflow for deploying *and using* a StorageClass is as follows:

1. Have a storage back-end (can be cloud or on premises)
2. Create your Kubernetes cluster
3. Install and configure the CSI storage plugin
4. Create one or more StorageClasses on Kubernetes

5. Deploy Pods and PVCs that reference those StorageClasses

The following YAML snippet defines a StorageClass, a PersistentVolumeClaim, and a Pod. All three objects can be defined in a single YAML file by separating them with three dashes (---).

Pay close attention to how the PodSpec references the PVC by name, and in turn, the PVC references the SC by name.

```
kind: StorageClass
apiVersion: storage.k8s.io/v1
metadata:
  name: fast                    <<=== Referenced by the PVC
provisioner: pd.csi.storage.gke.io
parameters:
  type: pd-ssd
---
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: mypvc                   <<=== Referenced by the PodSpec
spec:
  accessModes:
  - ReadWriteOnce
  resources:
    requests:
      storage: 50Gi
  storageClassName: fast        <<=== Matches name of the SC
---
apiVersion: v1
kind: Pod
metadata:
  name: mypod
spec:
  volumes:
    - name: data
      persistentVolumeClaim:
        claimName: mypvc        <<=== Matches PVC name
  containers: ...
  <SNIP>
```

The previous YAML is truncated and doesn't include a full PodSpec.

So far, you've seen a few SC definitions. However, each one was slightly different as each one related to a different provisioner (storage plugin/back-end). You'll need to refer to your plugin documentation to know which options yours supports.

Additional volume settings

There are a few other important settings you can configure in a StorageClass. We'll cover:

- Access mode
- Reclaim policy

Access mode

Kubernetes supports three access modes for volumes.

- `ReadWriteOnce` (RWO)
- `ReadWriteMany` (RWM)
- `ReadOnlyMany` (ROM)

`ReadWriteOnce` defines a PV that can only be bound as R/W by a single PVC. Attempts to bind it from multiple PVCs will fail.

`ReadWriteMany` defines a PV that can be bound as R/W by multiple PVCs. This mode is usually only supported by file and object storage such as NFS. Block storage usually only supports `RWO`.

`ReadOnlyMany` defines a PV that can be bound as R/O by multiple PVCs.

It's important to understand that a PV can only be opened in one mode – it's not possible for a single PV to be bound to a PVC in ROM mode and another PVC in RWM mode.

Reclaim policy

A volume's `ReclaimPolicy` tells Kubernetes how to deal with a PV when its PVC is released. Two policies currently exist:

- `Delete`
- `Retain`

`Delete` is the most dangerous, and it's the default for PVs created dynamically via *storage classes* unless you specify otherwise. It deletes the PV **and associated storage resource on the external storage system** when the PVC is released. This means all data will be lost! You should obviously use this policy with caution.

`Retain` will keep the associated `PV` object on the cluster as well as any data stored on the associated external asset. However, other PVCs are prevented from using it in future. The obvious disadvantage is it requires manual clean-up.

Let's quickly summarize what you've learned about storage classes before walking through a demo.

StorageClasses (SC) let you dynamically create physical back-end storage resources that get automatically mapped to Persistent Volumes (PV) on Kubernetes. You define SCs in YAML files that reference a plugin and tie them to a particular tier of storage on a particular storage back-end. For example, *high-performance AWS SSD storage in the AWS Mumbai Region.* The SC needs a name, and you deploy it using `kubectl apply`. Once deployed, the SC watches the API server for new PVC objects referencing its name. When matching PVCs appear, the SC dynamically creates the required asset on the back-end storage system and maps it to a PV on Kubernetes. Apps can then claim it with a PVC.

There's always more detail, such as *mount options* and *volume binding modes*, but what you've learned so far is enough to get you started.

Let's bring everything together with a demo.

Hands-on

This section walks you through using StorageClasses to dynamically provision storage on an external storage system and have it mapped to Kubernetes. We'll split the work as follows:

- Use an existing storage class
- Create and use a new storage class

The examples will only work on a *regional* Google Kubernetes Engine (GKE) cluster with the CSI plugin installed. If you created a GKE cluster as shown in Chapter 3, you're ready to go. If your Kubernetes cluster is somewhere else, the StorageClass YAML won't work, but the overall workflow will be the same.

Using an existing StorageClass

The following command lists all SCs defined on a typical GKE cluster. Yours may look different.

```
$ kubectl get sc
NAME                 PROVISIONER             RECLAIMPOLICY   VOLUMEBINDINGMODE
premium-rwo          pd.csi.storage.gke.io   Delete          WaitForFirstConsumer
standard (default)   kubernetes.io/gce-pd    Delete          Immediate
standard-rwo         pd.csi.storage.gke.io   Delete          WaitForFirstConsumer
```

There's quite a lot to learn from the output.

First up, all three SCs were automatically created when the cluster was built. This is common on hosted Kubernetes platforms, but your cluster may not have any.

The one on the second line is listed as the "default". This means it will be used by any PVCs that do not explicitly specify an SC. Default SCs are only useful in development environments and times when you do not have specific storage requirements. In production environments, you should explicitly use an SC that meets the requirements of the app.

The `PROVISIONER` column shows two of the SCs using the CSI plugin, the other is using the legacy in-tree plugin.

The `RECLAIM POLICY` is set to `Delete` for all three. This means any PVCs that use these SCs will create PVs and volumes that will be deleted when the PVC is deleted. This will result in data being lost. The alternative is `Retain`.

Setting `VOLUMEBINDINGMODE` to "Immediate" will create the volume on the external storage system as soon as the PVC is created. If you have multiple datacenters or cloud regions, the volume might be created in a different datacenter or region than the Pod that eventually consumes it. Setting it to `WaitForFirstConsumer` will delay creation until a Pod using the PVC is created. This ensures the volume will be created in the same datacenter or region as the Pod.

You can use `kubectl describe` to get more detailed information, and `kubectl get sc <name> -o yaml` will show the full configuration in YAML format.

```
$ kubectl describe sc premium-rwo
Name:                  premium-rwo
IsDefaultClass:        No
Annotations:           components.gke.io/component-name=pdcsi-addon...
Provisioner:           pd.csi.storage.gke.io
Parameters:            type=pd-ssd
AllowVolumeExpansion:  True
MountOptions:          <none>
ReclaimPolicy:         Delete
VolumeBindingMode:     WaitForFirstConsumer
Events:                <none>
```

Let's create a new volume using the `premium-rwo` SC.

List any existing PVs and PVCs so that it's easy to identify the ones you'll create in the next steps.

```
$ kubectl get pv
No resources found
$ kubectl get pvc
No resources found in default namespace.
```

The following PVC definition is from the `pvc-gke-premium.yml` file in the `storage` folder of the book's GitHub repo. It describes a PVC called `pvc-prem` that will provision a 10GB volume via the `premium-rwo` StorageClass. It will only work if your GKE cluster has a StorageClass called `premium-rwo`.

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: pvc-prem
spec:
  accessModes:
  - ReadWriteOnce
  storageClassName: premium-rwo
  resources:
    requests:
      storage: 10Gi
```

Create the PVC with the following command. Be sure to run it from the folder where the YAML file exists.

```
$ kubectl apply -f pvc-gke-premium.yml
persistentvolumeclaim/pvc-prem created
```

The following commands show a PVC has been created. However, it's in the pending state and no PV has been created. This is because the `premium-rwo` StorageClass volume binding mode is set to `WaitForFirstConsumer` meaning it won't provision a volume and PV until a Pod claims it.

```
$ kubectl get pv
No resources found
kubectl get pvc
NAME       STATUS    VOLUME   CAPACITY   ACCESS MODES   STORAGECLASS   AGE
pvc-prem   Pending                                      premium-rwo    1m
```

Create the Pod from the `prempod.yml` file. This Pod mounts a volume via the `pvc-prem` PersistentVolumeClaim that you just created and is currently in the `pending` state awaiting a Pod to mount it.

```
$ kubectl apply -f prempod.yml
pod/prempod created
```

Give the Pod a minute to start, then re-check the status of any PVCs and PVs.

```
$ kubectl get pvc
NAME       STATUS   VOLUME            CAPACITY   ACCESS MODES   STORAGECLASS   AGE
pvc-prem   Bound    pvc-796afda3...   10Gi       RWO            premium-rwo    9m48s

kubectl get pv
NAME           CAPACITY   MODES   RECLAIM POLICY   STATUS   CLAIM              STORAGECLASS
pvc-796af...   10Gi       RWO     Delete           Bound    default/pvc-prem   premium-rwo
```

The PVC now shows as bound and an associated PV has been created. If you check the Google Cloud backend, you'll see a new persistent disk created with the same name (see Google Cloud Console > Compute Engine > Disks).

Delete the Pod and the PVC.

```
$ kubectl delete pod prempod
pod "prempod" deleted

$ kubectl delete pvc pvc-prem
persistentvolumeclaim "pvc-prem" deleted
```

When the PVC is deleted, the PV and associated volume on the Google Cloud back-end will be automatically deleted. This is because the PVC was created through the `premium-rwo` SC which has the `ReclaimPolicy` set to `Delete`. Verify this.

```
$ kubectl get pv
No resources found
```

You can also check in the Google Cloud Console by going to `Compute Engine > Disks`.

Creating and using a new storage class

In this section, you'll create your own new storage class and use it to create a new volume.

The SC you'll create is defined in the `sc-gke-fast-repl.yml` file in the `storage` folder of the book's GitHub repo and defines a class called "sc-fast-repl" with the following properties.

- Fast SSD storage (`type: pd-ssd`)
- Replicated (`replication-type: regional-pd`)
- Create on-demand (`volumeBindingMode: WaitForFirstConsumer`)
- Keep data when the PVC is deleted (`reclaimPolicy: Retain`)

```
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: sc-fast-repl
provisioner: pd.csi.storage.gke.io
parameters:
  type: pd-ssd
  replication-type: regional-pd
volumeBindingMode: WaitForFirstConsumer
reclaimPolicy: Retain
```

Deploy the SC and verify it exists. You must run the command from the folder containing the YAML file, and it will only work on ***regional*** GKE clusters with the GKE CSI driver running.

```
$ kubectl apply -f sc-gke-fast-repl.yml
storageclass.storage.k8s.io/sc-fast-repl created

$ kubectl get sc
NAME                PROVISIONER             RECLAIMPOLICY   VOLUMEBINDINGMODE
premium-rwo         pd.csi.storage.gke.io   Delete          WaitForFirstConsumer
sc-fast-repl        pd.csi.storage.gke.io   Retain          WaitForFirstConsumer
...
```

With the SC in place, deploy the app and PVC defined in the `vol-app.yml` file. It contains a PVC defining a 20G volume based on the newly created `sc-fast-repl` SC. It also defines a Pod that mounts a volume from it. When you deploy the app, a new PVC and PV will be created, as well as a replicated persistent disk on the Google Cloud back-end.

```
$ kubectl apply -f vol-app.yml
persistentvolumeclaim/pvc1 created
pod/volpod created
```

Use `kubectl` to check the PVC and PV exist.

The mechanics behind the operation that created the PV are as follows:

1. You created the `sc-fast-repl` StorageClass
2. The SC controller started watching the API server for new PVCs referencing it
3. The app you deployed created the `pvc2` PVC that requested a 20GB volume from the `sc-fast-repl` *StorageClass*
4. The StorageClass loop noticed the PVC and dynamically created the back-end volume and PV

Congratulations. You've seen how to create a StorageClass and dynamically create volumes from it.

Clean-up

The Pod and PVC were both deployed from the `vol-app.yml` file. This means you can delete them both with the following command.

```
$ kubectl delete -f vol-app.yml
persistentvolumeclaim "pvc2" deleted
pod "volpod" deleted
```

Even though the Pod and PVC are deleted, a `kubectl get pv` will show the PV still exists. This is because the class it was created from is using the `Retain` reclaim policy. This keeps PVs, associated back-end volumes, **and data** even when PVCs are deleted. You can verify this in the Google Cloud back-end (`Compute Engine > Disks` and check for a 20G regional disk).

Manually delete the PV with a `kubectl delete pv` command and then be sure to delete the regional disk on the Google Cloud back-end. Failure to delete the regional disk on the back-end may result in unexpected charges.

Chapter Summary

In this chapter, you learned that Kubernetes has a powerful storage subsystem that allows it to dynamically provision and leverage storage from a wide variety of external storage back-ends.

Each back-end requires a plugin to expose its assets to Kubernetes, and the preferred type of plugin is a CSI plugin. Once a plugin is enabled, Persistent Volumes (PV) are used to represent external storage resources on the Kubernetes cluster, and Persistent Volume Claims (PVC) are used to give Pods access to PV storage.

Storage Classes allow applications to dynamically request storage. You create a Storage Class object that references a class, or tier, of storage from a storage back-end. Once created, the Storage Class watches the API Server for new Persistent Volume Claims that reference it. When a matching PVC arrives, the SC dynamically creates the storage and makes it available as a PV that can be mounted as a volume into a Pod (container).